DS4420 Final Project Bayes Model

April 15 2026

```
# download required libraries
library(dplyr)
library(ggplot2)
library(tidyr)
```

```
# load our dataset (daily training data)
df <- read.csv("run_ww_2019_d.csv")
glimpse(df)
```

```
## Rows: 13,290,380
## Columns: 9
## $ X         <int> 0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17…
## $ datetime  <chr> "2019-01-01", "2019-01-01", "2019-01-01", "2019-01-01", "201…
## $ athlete   <int> 0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17…
## $ distance  <dbl> 0.00, 5.27, 0.00, 10.50, 9.66, 10.38, 10.11, 0.00, 9.99, 0.0…
## $ duration  <dbl> 0.00000, 30.20000, 0.00000, 43.95000, 48.65000, 50.13333, 53…
## $ gender    <chr> "F", "M", "M", "M", "M", "F", "M", "M", "M", "M", "M", "M", …
## $ age_group <chr> "18 - 34", "35 - 54", "35 - 54", "18 - 34", "35 - 54", "35 -…
## $ country   <chr> "United States", "Germany", "United Kingdom", "United Kingdo…
## $ major     <chr> "CHICAGO 2019", "BERLIN 2016", "LONDON 2018,LONDON 2019", "L…
```

```r
# drop rest days, aggregate training stats per athlete
features <- df %>%
  filter(distance > 0) %>%
  group_by(athlete) %>%
  summarise(
    avg_weekly_km = mean(distance, na.rm = TRUE),
    total_km      = sum(distance, na.rm = TRUE),
    avg_pace      = mean(duration / distance, na.rm = TRUE),
    sessions      = n(),
    age_group     = first(age_group),
    gender        = first(gender)
  )

# one marathon per athlete, full marathon distance only (40-45km to include GPS error)
marathon_rows <- df %>%
  filter(major != "" & !is.na(major) & duration > 0 & distance >= 40 & distance <= 45) %>%
  group_by(athlete) %>%
  slice_max(duration, n = 1, with_ties = FALSE) %>%
  ungroup() %>%
  select(athlete, finish_time = duration)

# join and filter to reasonable finish time range
model_data <- features %>%
  inner_join(marathon_rows, by = "athlete") %>%
  filter(finish_time > 120, finish_time < 600) %>%
  drop_na()

glimpse(model_data)
```

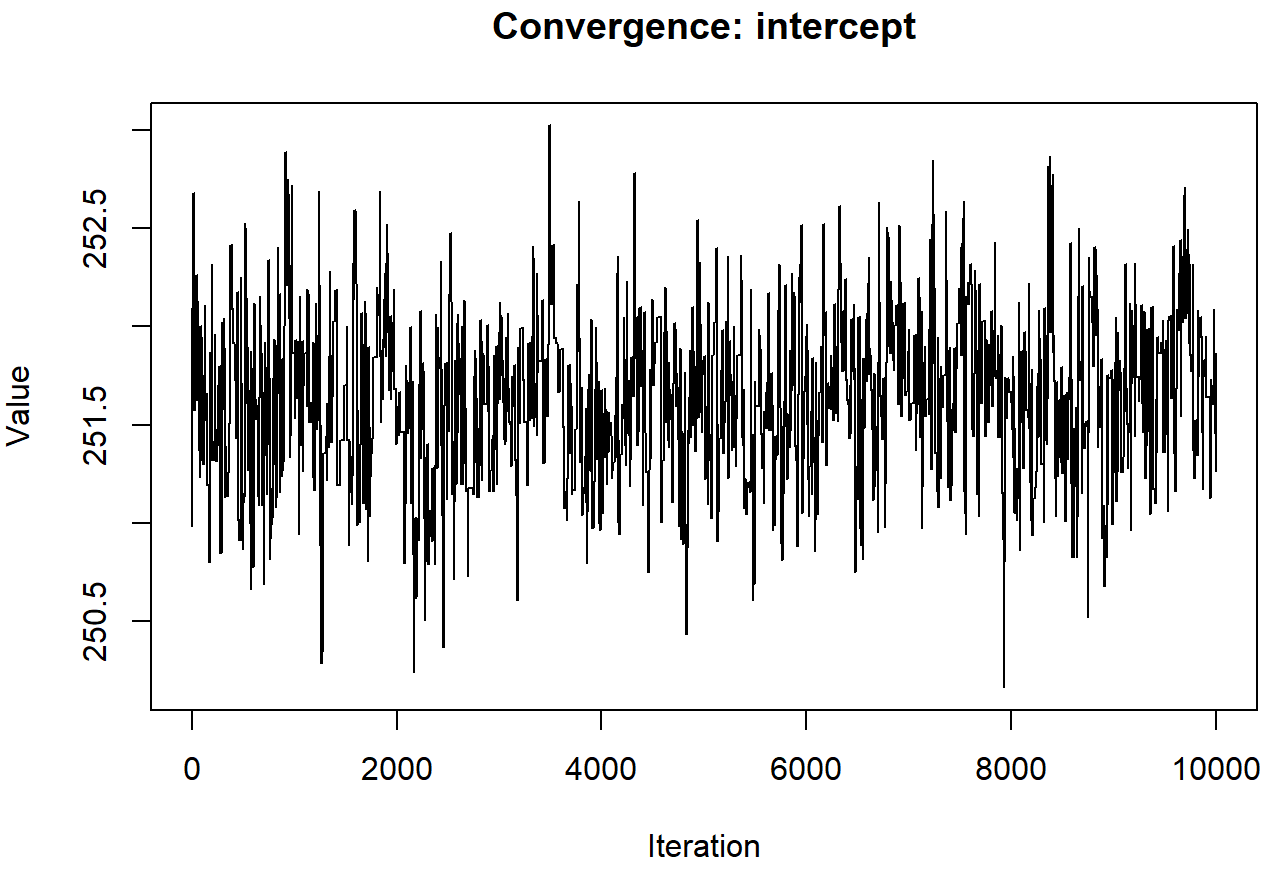
```
## Rows: 24,693
## Columns: 8
## $ athlete       <int> 0, 1, 2, 4, 6, 8, 9, 11, 12, 13, 14, 15, 16, 18, 20, 22,…
## $ avg_weekly_km <dbl> 11.031818, 12.206555, 10.900284, 10.059776, 14.339979, 9…
## $ total_km      <dbl> 606.750, 2770.888, 1035.527, 1166.934, 3412.915, 886.186…
## $ avg_pace      <dbl> 6.074183, 5.909987, 6.894187, 5.069657, 5.606458, 5.7993…
## $ sessions      <int> 55, 227, 95, 116, 238, 97, 34, 126, 136, 87, 268, 193, 1…
## $ age_group     <chr> "18 - 34", "35 - 54", "35 - 54", "35 - 54", "55 +", "35 …
## $ gender        <chr> "F", "M", "M", "M", "M", "M", "M", "M", "M", "M", "M", "…
## $ finish_time   <dbl> 283.0, 270.0, 237.0, 217.0, 237.0, 239.0, 282.0, 237.0, …
```

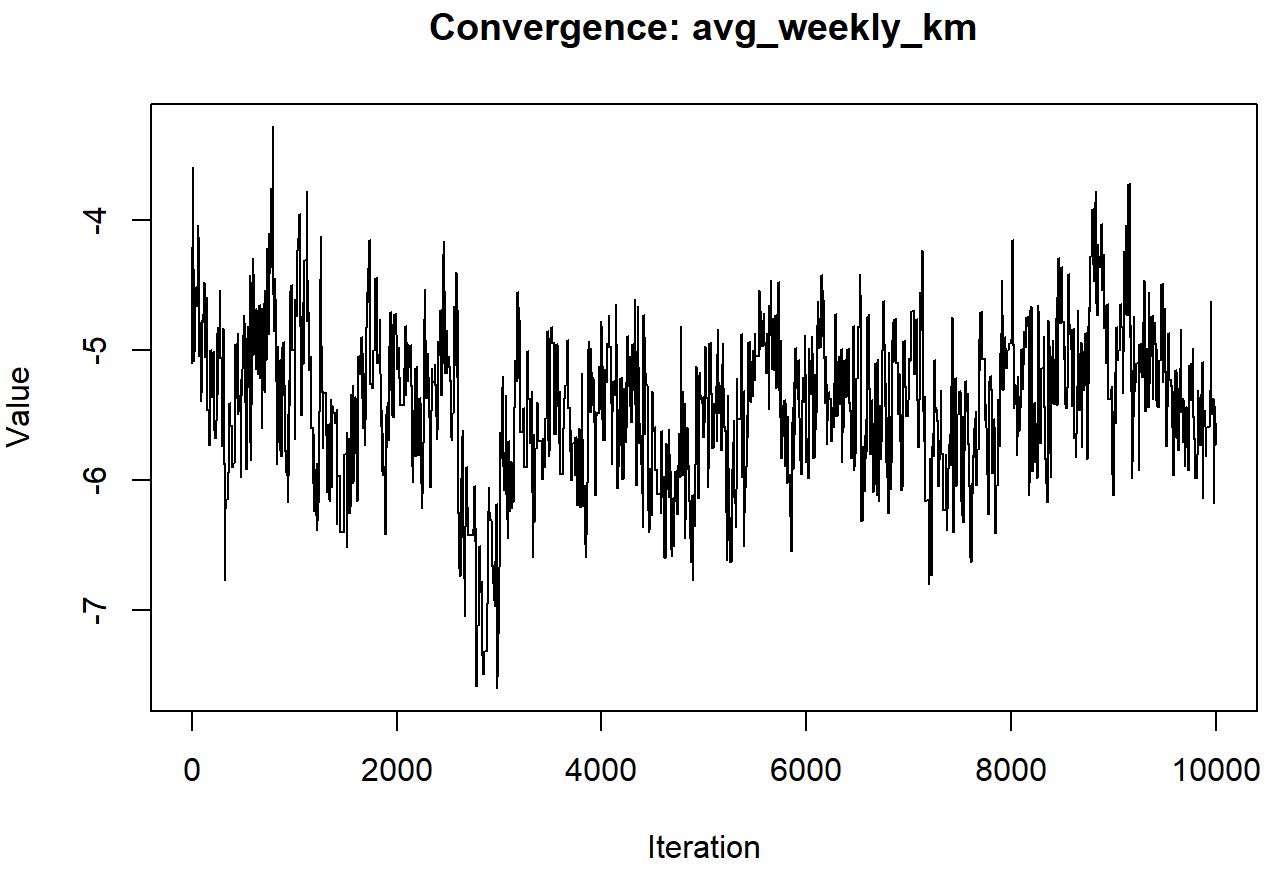
```r
# encode categorical features
model_data <- model_data %>%
  mutate(
    gender_m   = as.integer(gender == "M"),
    age_35_54  = as.integer(age_group == "35 - 54"),
    age_55plus = as.integer(age_group == "55 +")
  )

# 80/20 train/test split
set.seed(42)
idx        <- sample(nrow(model_data), floor(0.8 * nrow(model_data)))
train_data <- model_data[idx, ]
test_data  <- model_data[-idx, ]

# scale features using training data, add intercept col
train_matrix <- train_data %>%
  select(avg_weekly_km, total_km, avg_pace, sessions, gender_m, age_35_54, age_55plus)
train_means  <- colMeans(train_matrix)
train_sds    <- apply(train_matrix, 2, sd)

X <- scale(train_matrix) %>% cbind(intercept = 1, .)
y <- train_data$finish_time

X_test <- sweep(sweep(test_data %>%
  select(avg_weekly_km, total_km, avg_pace, sessions, gender_m, age_35_54, age_55plus),
  2, train_means, "-"), 2, train_sds, "/") %>%
  cbind(intercept = 1, .)
y_test <- test_data$finish_time
```

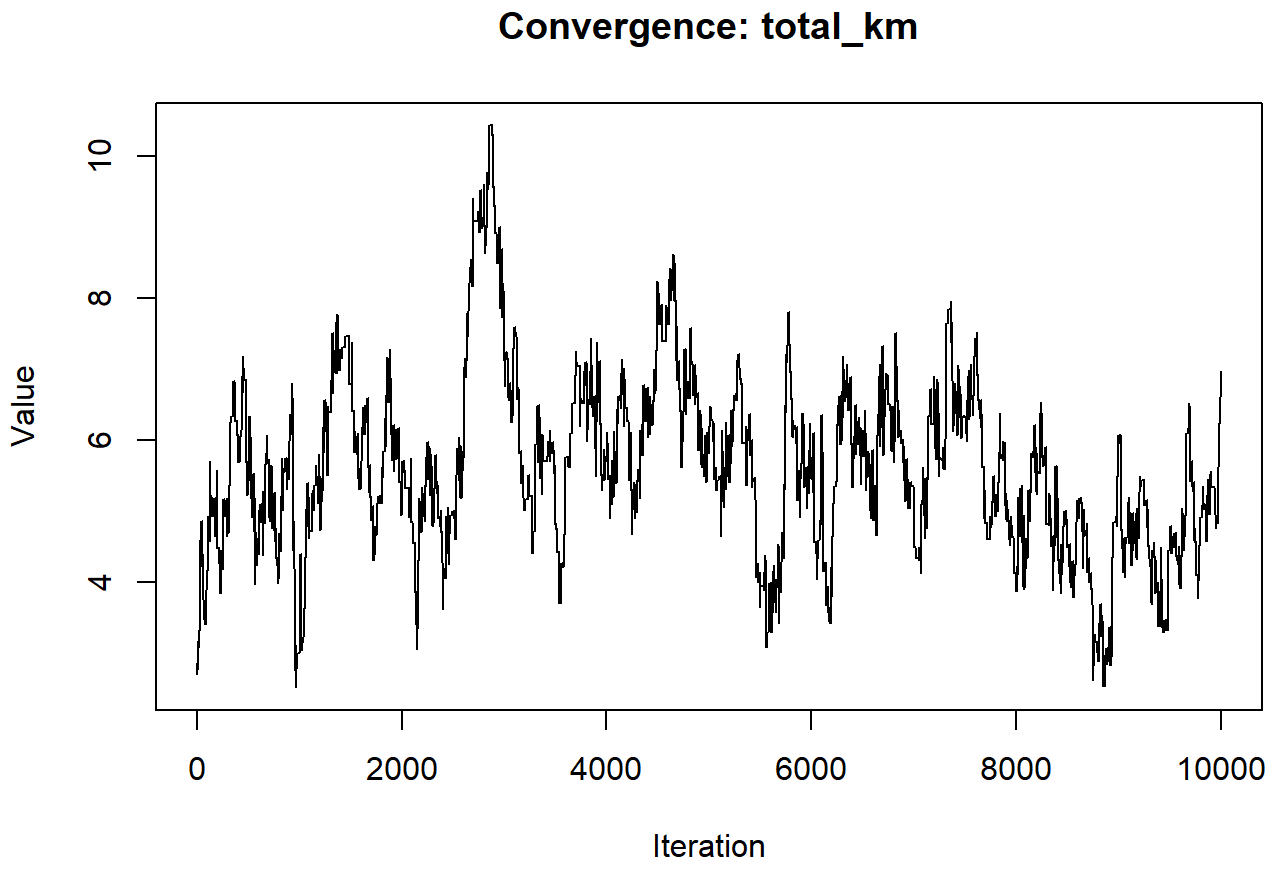
```r
set.seed(42)

# log posterior: gaussian likelihood + priors on beta and sigma
# intercept ~ N(240, 60), coefficients ~ N(0, 10), sigma ~ N(0, 20)
log_posterior <- function(beta, sigma, X, y) {
  if (sigma <= 0) return(-Inf)
  mu   <- X %*% beta
  ll   <- sum(dnorm(y, mean = mu, sd = sigma, log = TRUE))
  lp_b <- sum(dnorm(beta[-1], 0, 10, log = TRUE))
  lp_b <- lp_b + dnorm(beta[1], 240, 60, log = TRUE)
  lp_s <- dnorm(sigma, 0, 20, log = TRUE)
  return(ll + lp_b + lp_s)
}

# metropolis-hastings sampler, isotropic gaussian proposals
metropolis <- function(n, X, y, prop_sd = 1.5, burn = 0.2) {
  p            <- ncol(X)
  total        <- floor(n / (1 - burn))
  m            <- floor(total * burn)
  beta         <- rep(0, p)
  sigma        <- 30
  samples_beta <- matrix(0, nrow = total, ncol = p)
  samples_sig  <- numeric(total)
  accepted     <- 0

  for (i in 1:total) {
    beta_prop  <- rnorm(p, beta, prop_sd)
    sigma_prop <- abs(rnorm(1, sigma, prop_sd))
    # acceptance ratio in log space for numerical stability
    log_alpha  <- log_posterior(beta_prop, sigma_prop, X, y) -
                  log_posterior(beta,      sigma,      X, y)
    if (log(runif(1)) < log_alpha) {
      beta     <- beta_prop
      sigma    <- sigma_prop
      accepted <- accepted + 1
    }
    samples_beta[i, ] <- beta
    samples_sig[i]    <- sigma
  }

  cat("Acceptance rate:", round(accepted / total, 3), "\n")
  # discard burn-in before returning
  list(
    beta  = samples_beta[(m + 1):total, ],
    sigma = samples_sig[(m + 1):total]
  )
}

trace <- metropolis(n = 10000, X = X, y = y, prop_sd = .4)
```

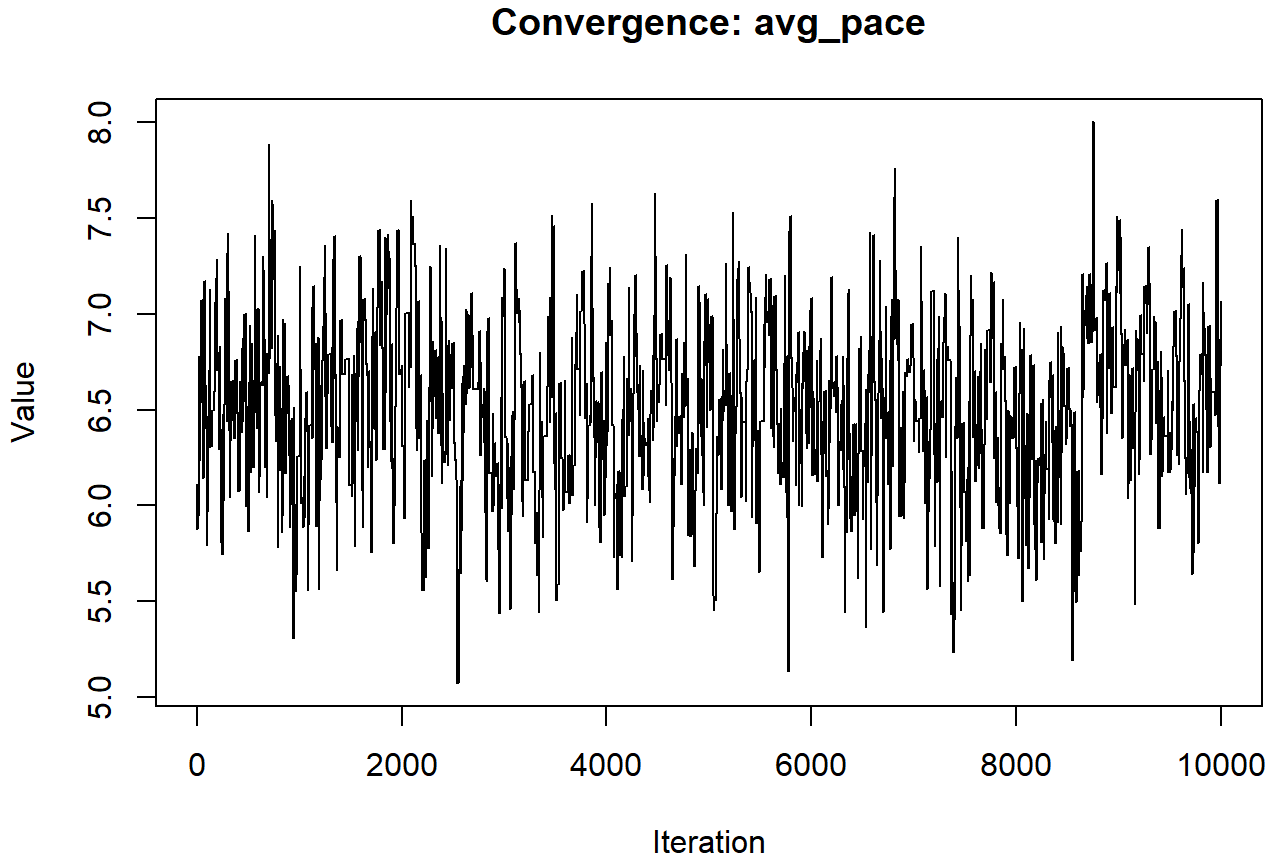
```
## Acceptance rate: 0.232
```

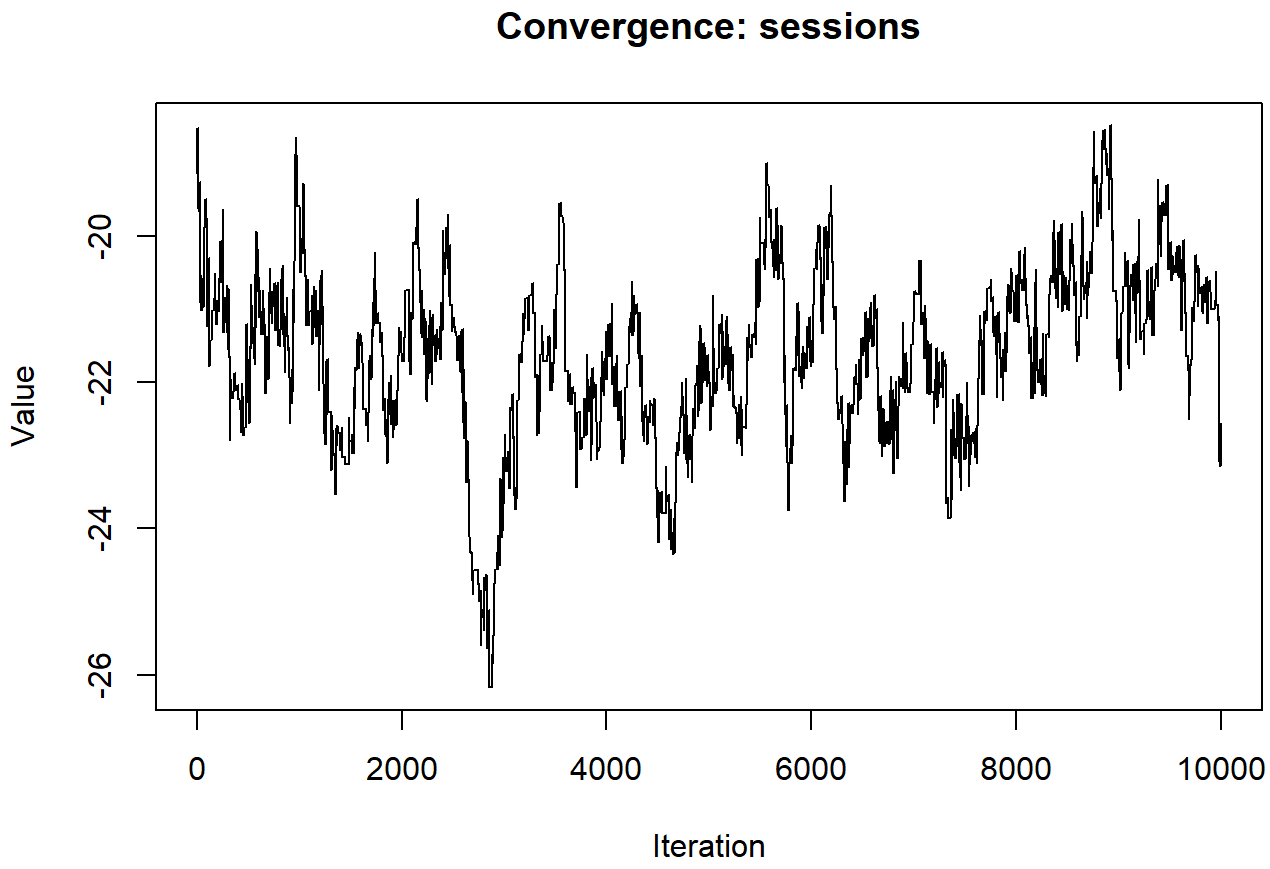
```r
par_names <- c("intercept", "avg_weekly_km", "total_km", "avg_pace",
               "sessions", "gender_m", "age_35_54", "age_55plus")

# check convergence and mixing for each parameter
for (i in 1:ncol(trace$beta)) {
  plot(trace$beta[, i], type = "l",
       main = paste("Convergence:", par_names[i]),
       xlab = "Iteration", ylab = "Value")
}
```

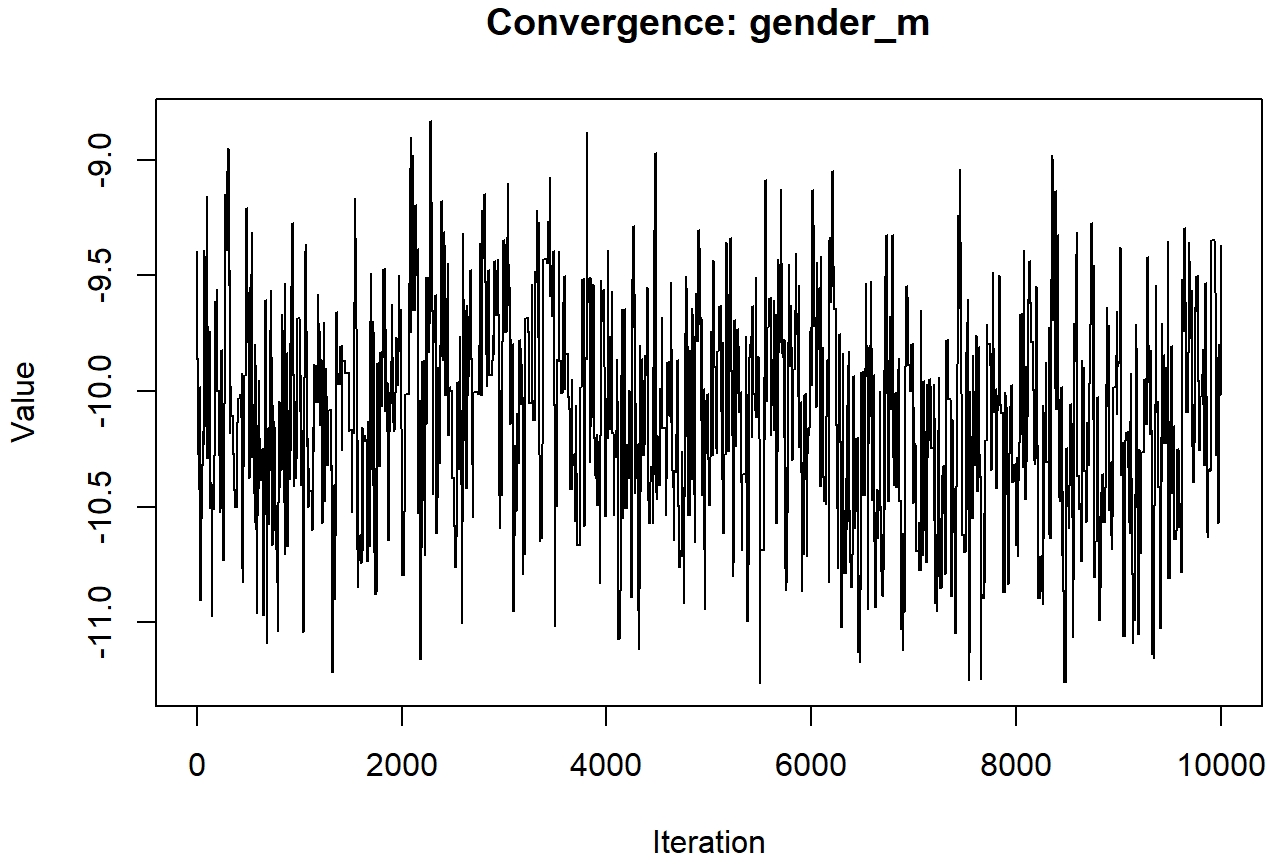

Convergence: intercept
Value
250.5
251.5
252.5
0
2000
4000
6000
8000
10000
Iteration

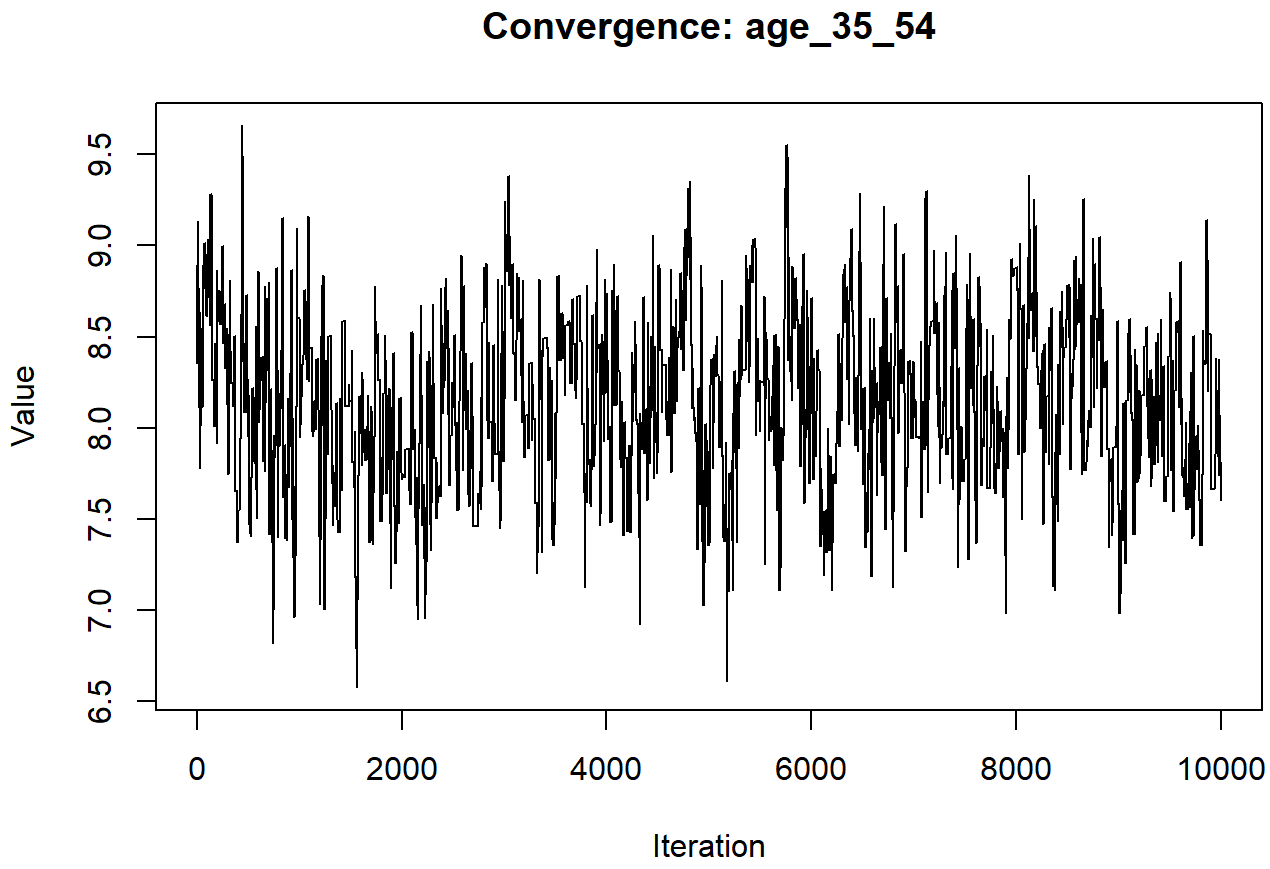

Convergence: avg_weekly_km
Value
-4
-5
-6
-7
0
2000
4000
6000
8000
10000
Iteration

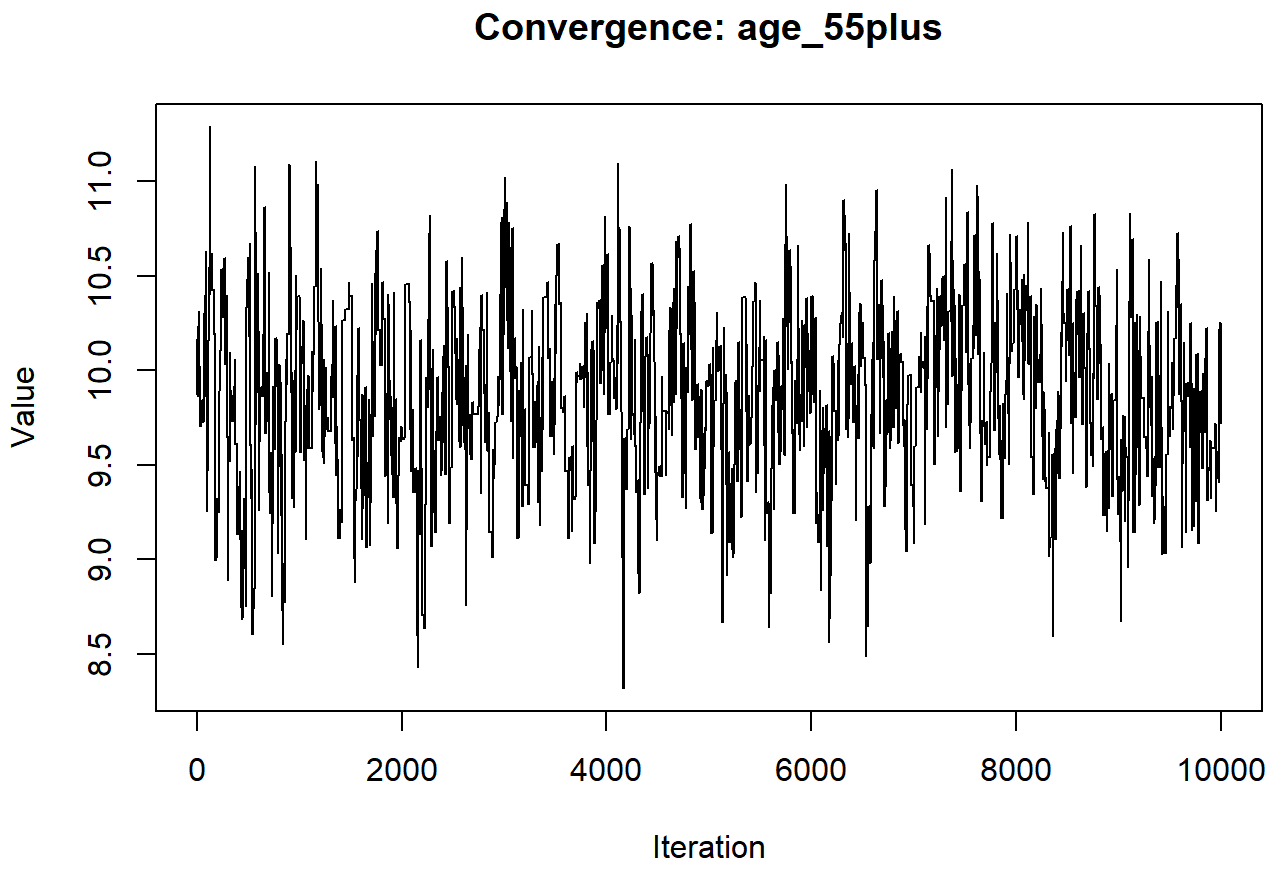

Convergence: total_km
Value
4
6
8
10
0
2000
4000
6000
8000
10000
Iteration


Convergence: avg_pace
Value
5.0
5.5
6.0
6.5
7.0
7.5
8.0
0
2000
4000
6000
8000
10000
Iteration

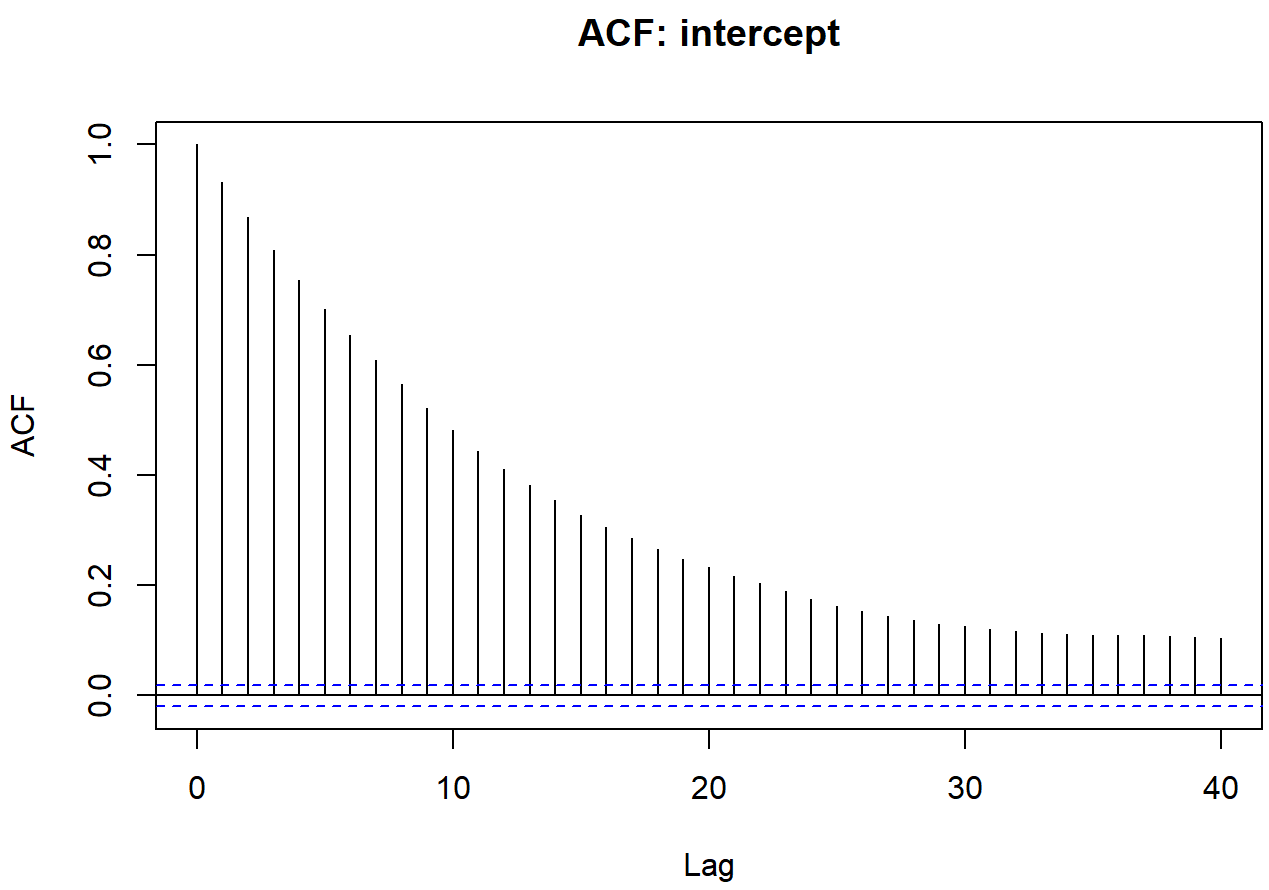

Convergence: sessions
Value
-20
-22
-24
-26
0
2000
4000
6000
8000
10000
Iteration

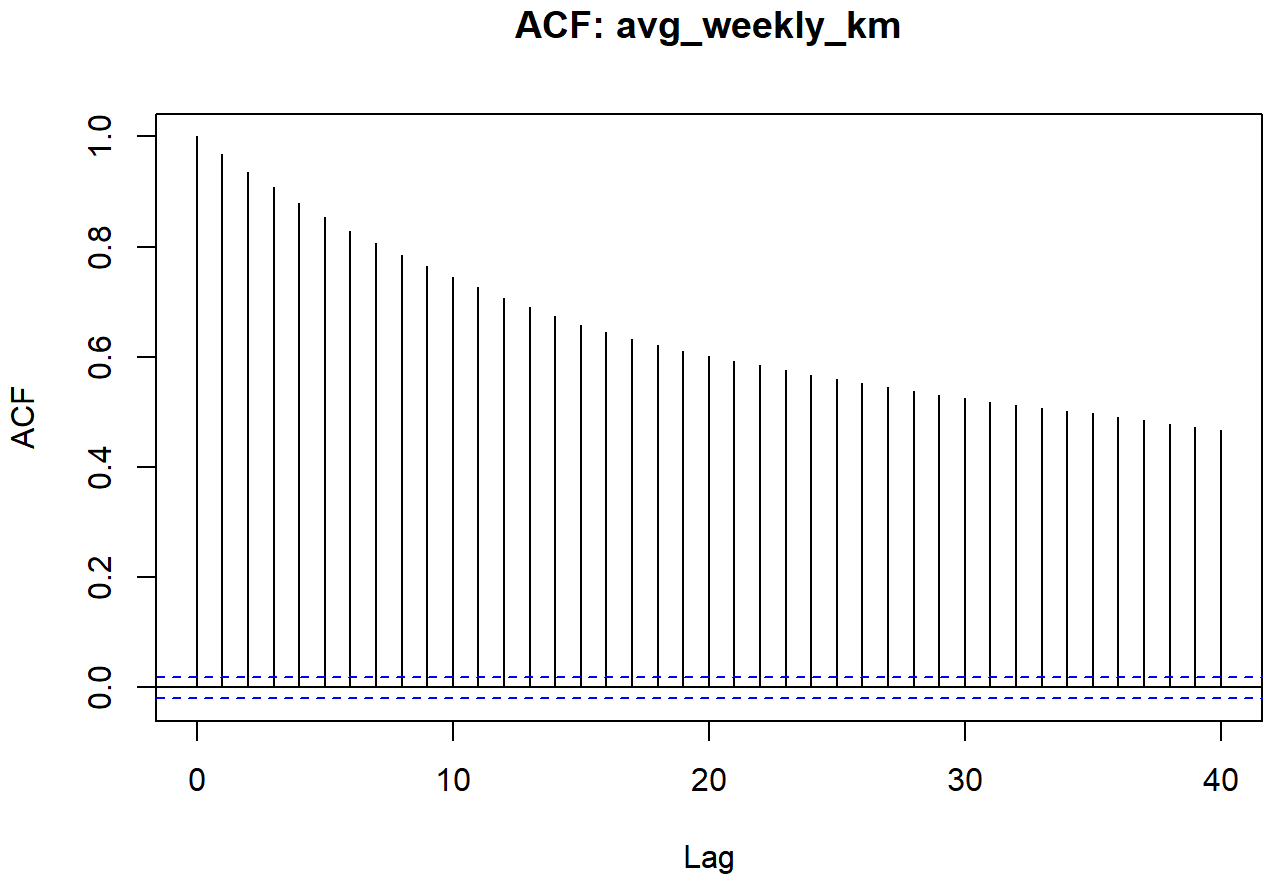

Convergence: gender_m
Value
-9.0
-9.5
-10.0
-10.5
-11.0
0
2000
4000
6000
8000
10000
Iteration

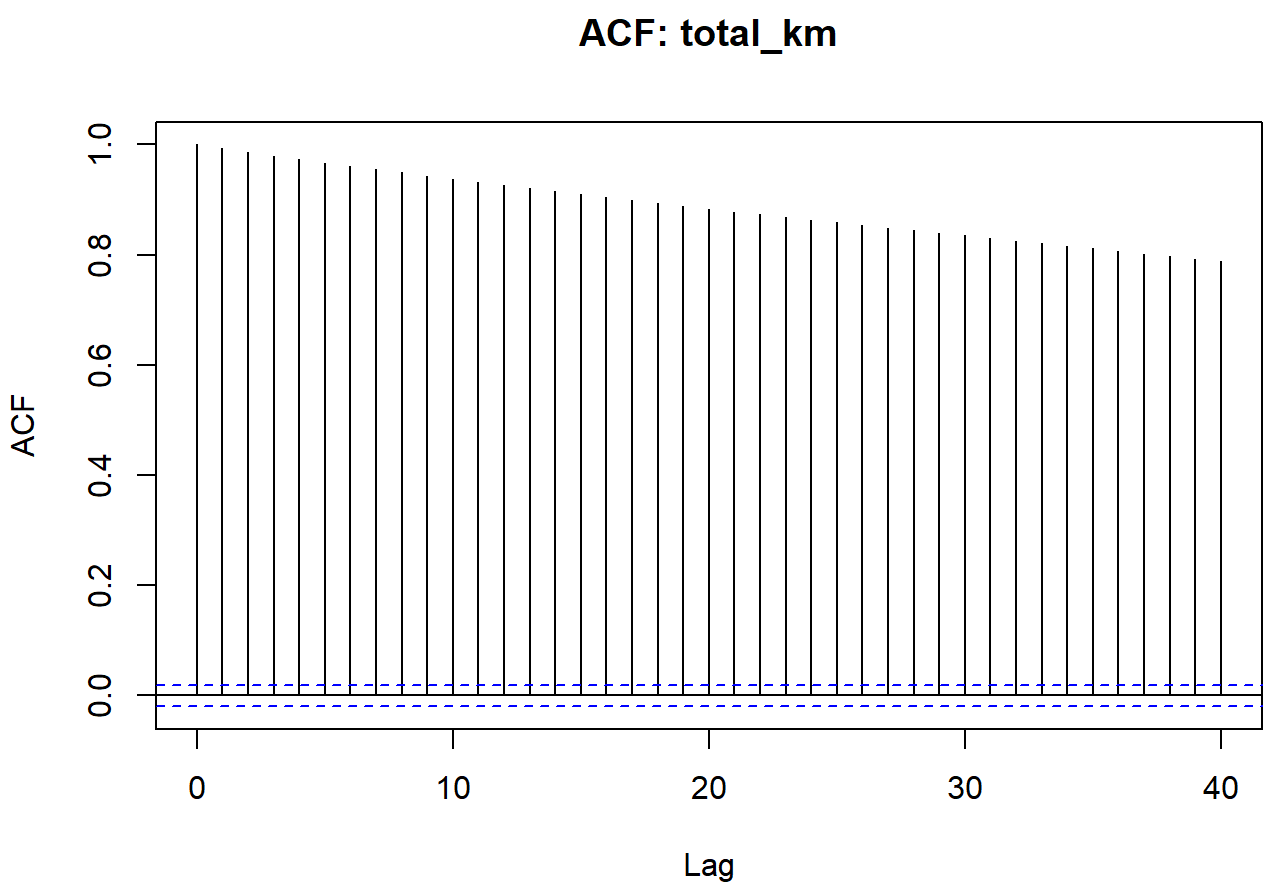

Convergence: age_35_54
Value
6.5
7.0
7.5
8.0
8.5
9.0
9.5
0
2000
4000
6000
8000
10000
Iteration

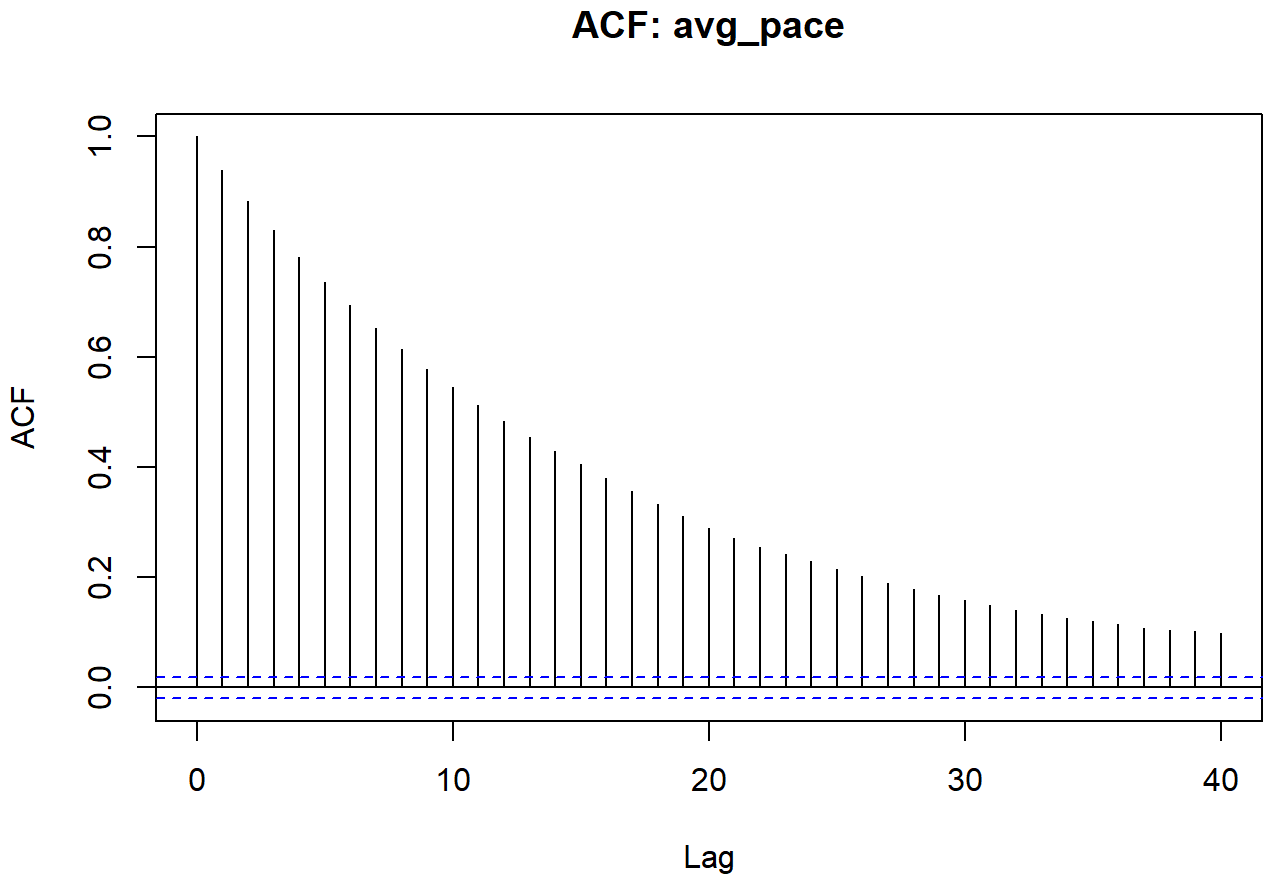

Convergence: age_55plus
Value
8.5
9.0
9.5
10.0
10.5
11.0
0
2000
4000
6000
8000
10000
Iteration

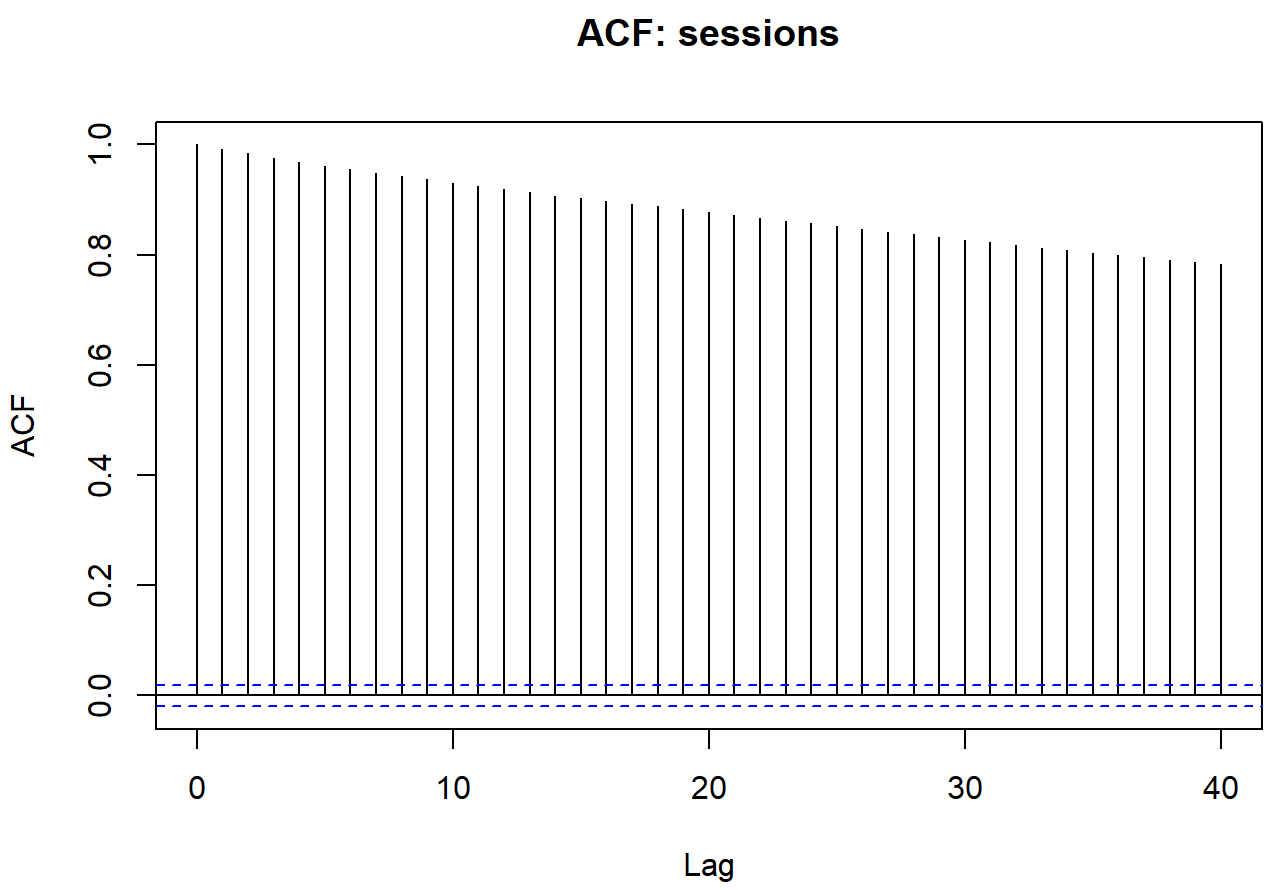
```r
for (i in 1:ncol(trace$beta)) {
  acf(trace$beta[, i], main = paste("ACF:", par_names[i]))
}
```

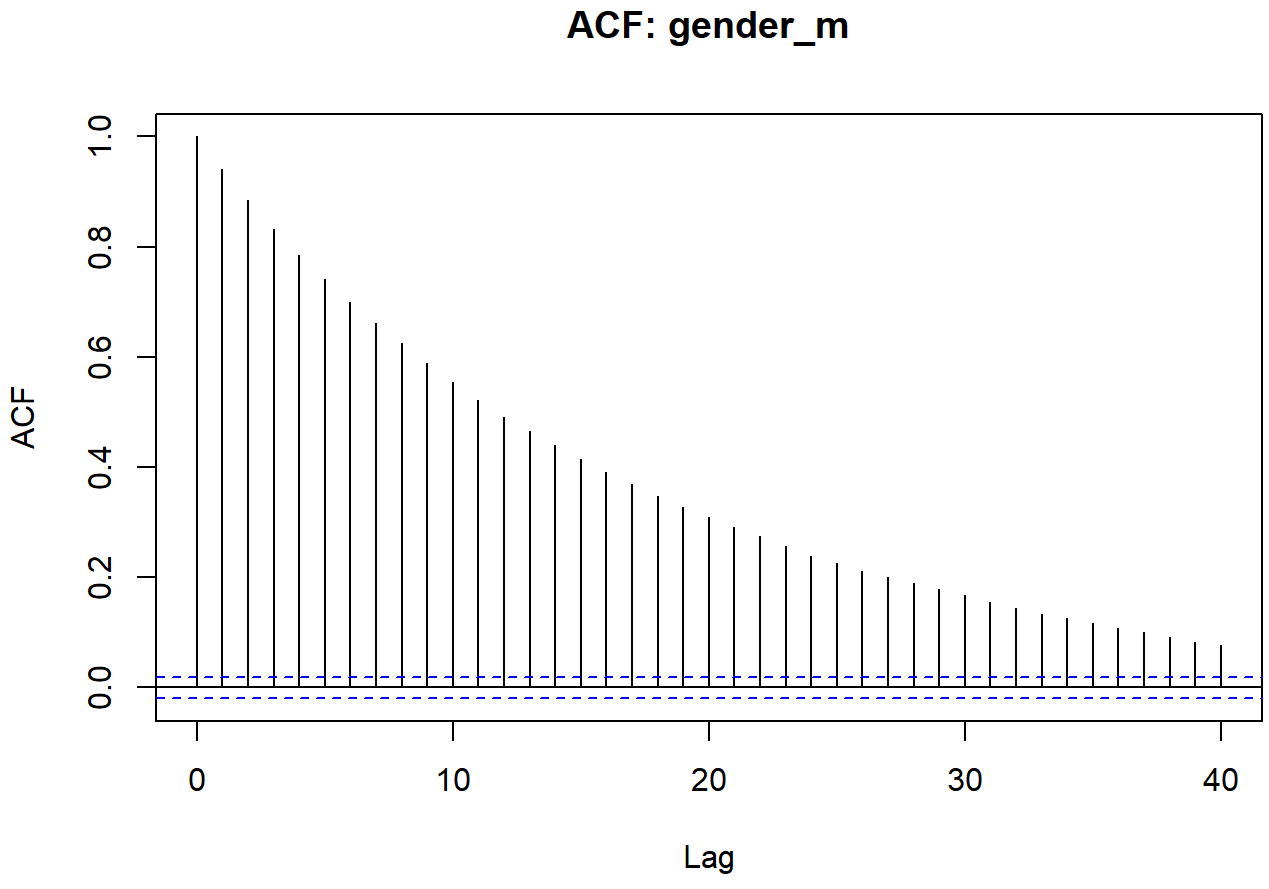

ACF: intercept
ACF
0.0
0.2
0.4
0.6
0.8
1.0
0
10
20
30
40
Lag

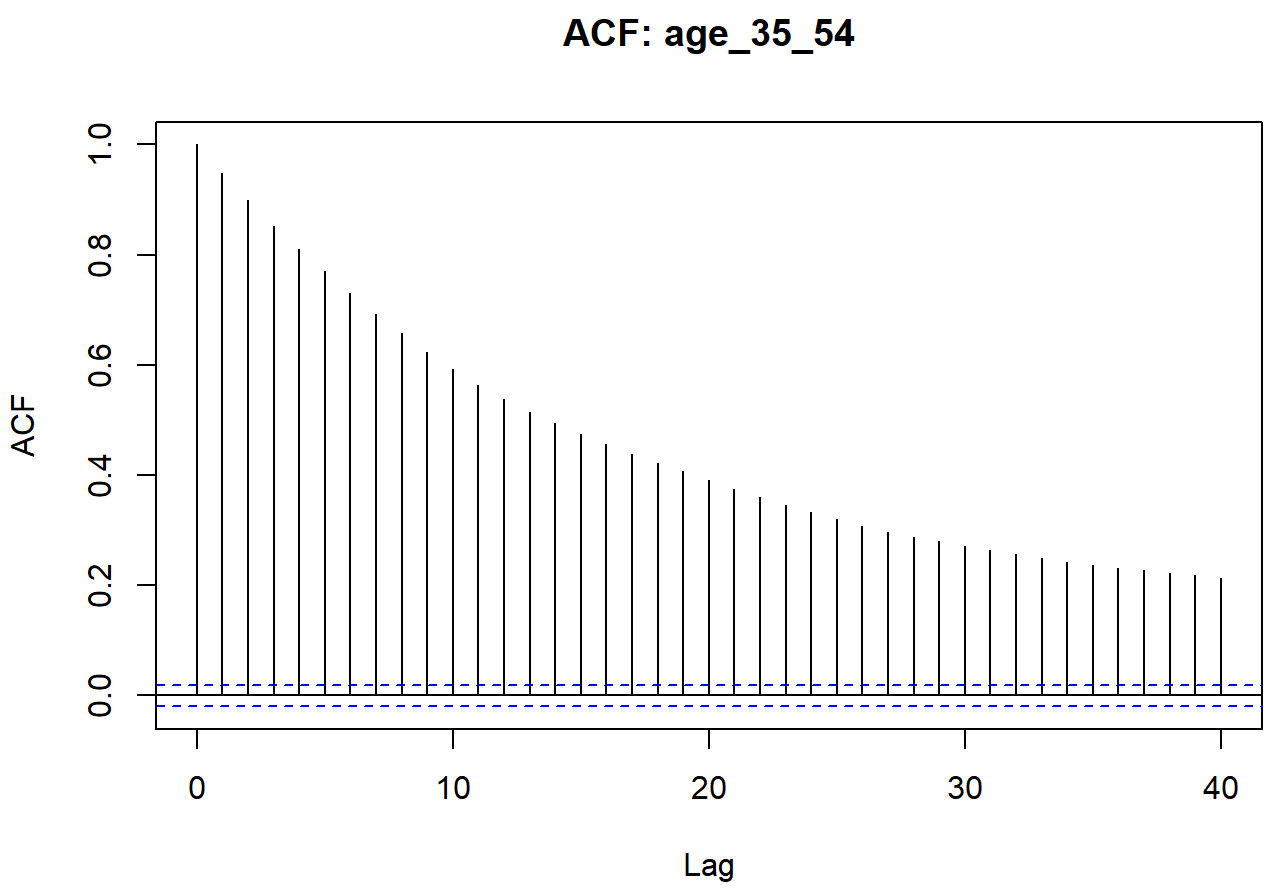

ACF: avg_weekly_km
ACF
0.0
0.2
0.4
0.6
0.8
1.0
0
10
20
30
40
Lag

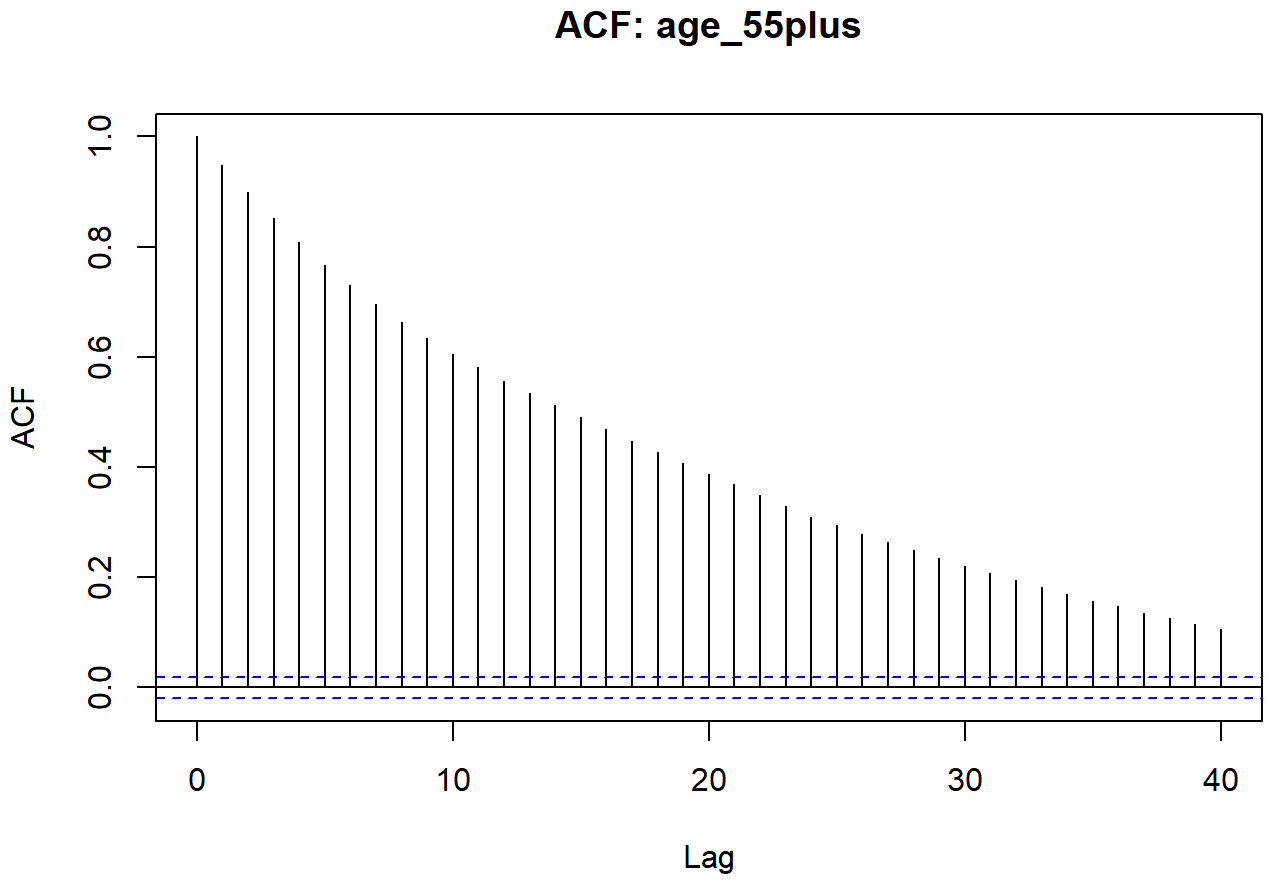

ACF: total_km
ACF
0.0
0.2
0.4
0.6
0.8
1.0
0
10
20
30
40
Lag


ACF: avg_pace
ACF
0.0
0.2
0.4
0.6
0.8
1.0
0
10
20
30
40
Lag


ACF: sessions
ACF
0.0
0.2
0.4
0.6
0.8
1.0
0
10
20
30
40
Lag


ACF: gender_m
ACF
0.0
0.2
0.4
0.6
0.8
1.0
0
10
20
30
40
Lag

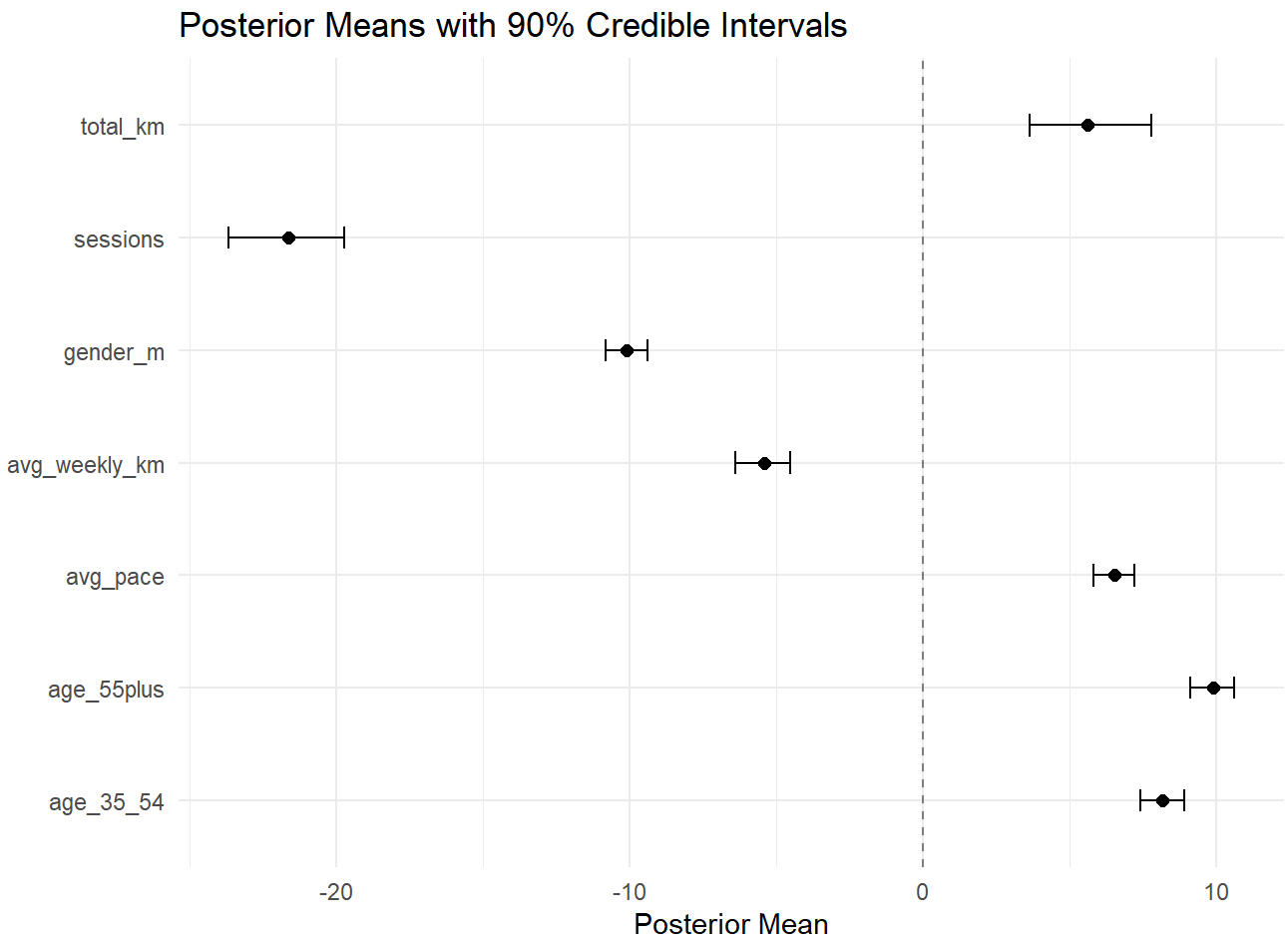

ACF: age_35_54
ACF
0.0
0.2
0.4
0.6
0.8
1.0
0
10
20
30
40
Lag


ACF: age_55plus
ACF
0.0
0.2
0.4
0.6
0.8
1.0
0
10
20
30
40
Lag

```
post_mean <- colMeans(trace$beta)
post_ci   <- apply(trace$beta, 2, quantile, probs = c(0.05, 0.95))

#summarize posterior
summary_df <- data.frame(
  parameter = par_names,
  mean      = round(post_mean, 3),
  ci_5      = round(post_ci[1, ], 3),
  ci_95     = round(post_ci[2, ], 3)
)

print(summary_df)
```

```
##       parameter    mean    ci_5   ci_95
## 1     intercept 251.640 250.988 252.315
## 2 avg_weekly_km  -5.421  -6.386  -4.544
## 3      total_km   5.613   3.636   7.778
## 4      avg_pace   6.512   5.795   7.207
## 5      sessions -21.623 -23.674 -19.742
## 6      gender_m -10.106 -10.811  -9.397
## 7     age_35_54   8.150   7.399   8.893
## 8    age_55plus   9.867   9.114  10.580
```

```
#plot of coefficients
ggplot(summary_df[-1, ], aes(x = mean, y = parameter)) +
  geom_point(size = 2) +
  geom_errorbarh(aes(xmin = ci_5, xmax = ci_95), height = 0.2) +
  geom_vline(xintercept = 0, linetype = "dashed", color = "gray50") +
  labs(title = "Posterior Means with 90% Credible Intervals",
       x = "Posterior Mean", y = "") +
  theme_minimal()
```




```
world_record <- 120.6

# sample up to 100 test runners to keep runtime manageable
set.seed(42)
test_idx  <- sample(nrow(X_test), min(100, nrow(X_test)))
X_test_s  <- X_test[test_idx, ]
y_test_s  <- y_test[test_idx]

# posterior predictive median for each test runner
test_preds <- apply(X_test_s, 1, function(xrow) {
  draws <- apply(trace$beta, 1, function(b) rnorm(1, mean = sum(xrow * b), sd = mean(trace$sigm
a)))
  draws <- draws[draws >= world_record]
  median(draws)
})

rmse <- sqrt(mean((test_preds - y_test_s)^2))
cat("Test RMSE (min):", round(rmse, 2), "\n")
```

```
## Test RMSE (min): 54.95
```

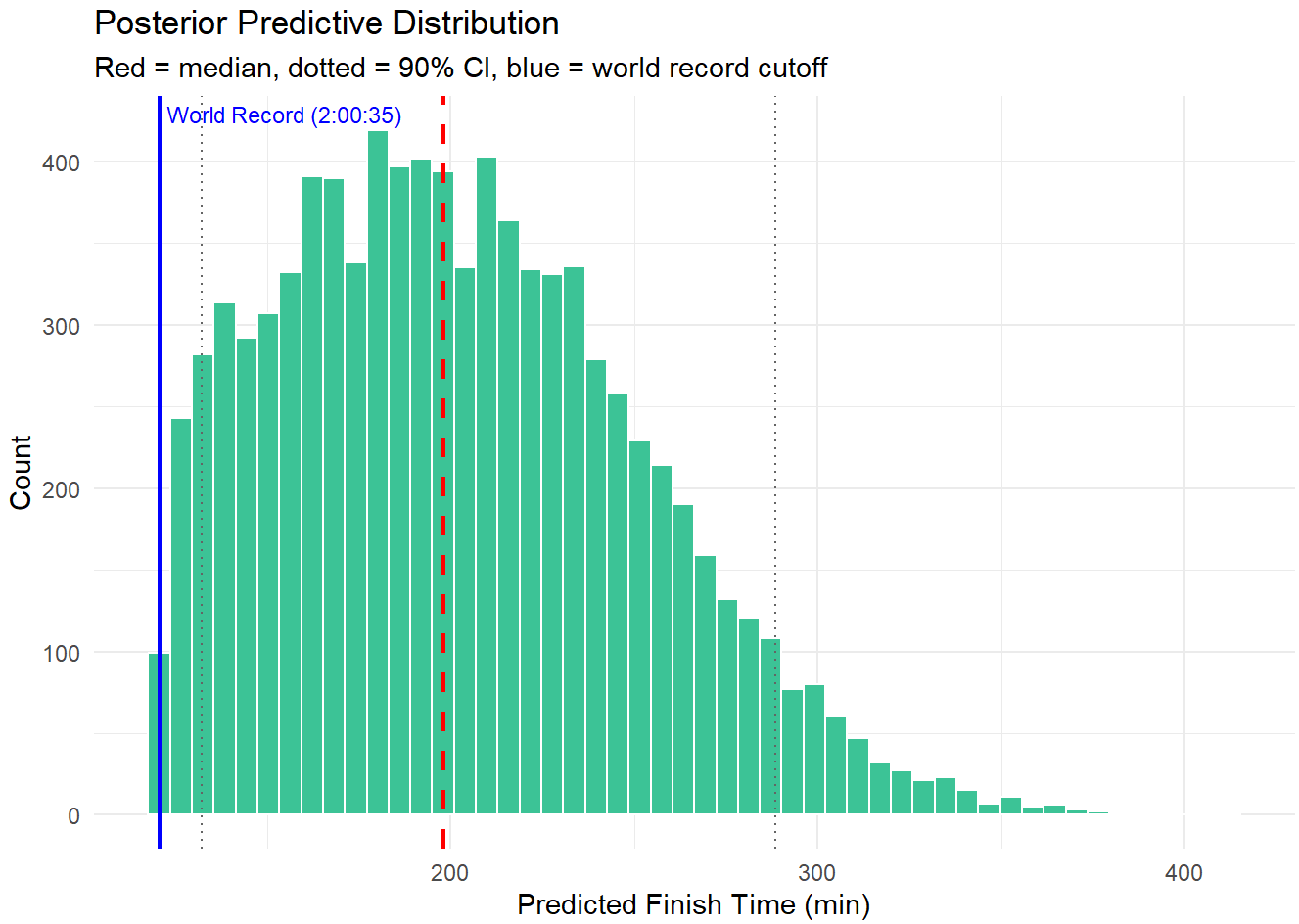
```
new_runner <- data.frame(
  avg_weekly_km = 55,
  total_km      = 800,
  avg_pace      = 6.2,
  sessions      = 80,
  gender_m      = 1,
  age_35_54     = 1,
  age_55plus    = 0
)

# scale new runner using training data parameters
new_scaled <- (new_runner - train_means) / train_sds
new_x      <- c(1, as.numeric(new_scaled))

# posterior predictive: one sample per posterior draw
preds <- apply(trace$beta, 1, function(b) rnorm(1, mean = new_x %*% b, sd = mean(trace$sigma)))

# drop anything below world record — kelvin kiptum, chicago 2023 (2:00:35)
preds <- preds[preds >= world_record]

cat("Median predicted finish time (min):", round(median(preds), 1), "\n")
```

```
## Median predicted finish time (min): 197.8
```

```
cat("90% CI:", round(quantile(preds, 0.05), 1), "-", round(quantile(preds, 0.95), 1), "\n")
```

```
## 90% CI: 132.1 - 288.5
```

```
# generate prediction plot - include a cutoff point at the world record time (normal distro woul
d have unrealistic time constraints)
data.frame(finish_time = preds) %>%
  ggplot(aes(x = finish_time)) +
  geom_histogram(bins = 50, fill = "#10B981", color = "white", alpha = 0.8) +
  geom_vline(xintercept = median(preds),         color = "red",    linewidth = 1, linetype = "da
shed") +
  geom_vline(xintercept = quantile(preds, 0.05), color = "gray40", linetype = "dotted") +
  geom_vline(xintercept = quantile(preds, 0.95), color = "gray40", linetype = "dotted") +
  geom_vline(xintercept = world_record,          color = "blue",   linewidth = 0.8, linetype =
"solid") +
  annotate("text", x = world_record + 2, y = Inf, label = "World Record (2:00:35)",
           color = "blue", hjust = 0, vjust = 1.5, size = 3) +
  labs(title = "Posterior Predictive Distribution",
       subtitle = "Red = median, dotted = 90% CI, blue = world record cutoff",
       x = "Predicted Finish Time (min)", y = "Count") +
  theme_minimal()
```


Posterior Predictive Distribution
Red = median, dotted = 90% CI, blue = world record cutoff
World Record (2:00:35)
Count
0
100
200
300
400
200
300
400
Predicted Finish Time (min)